

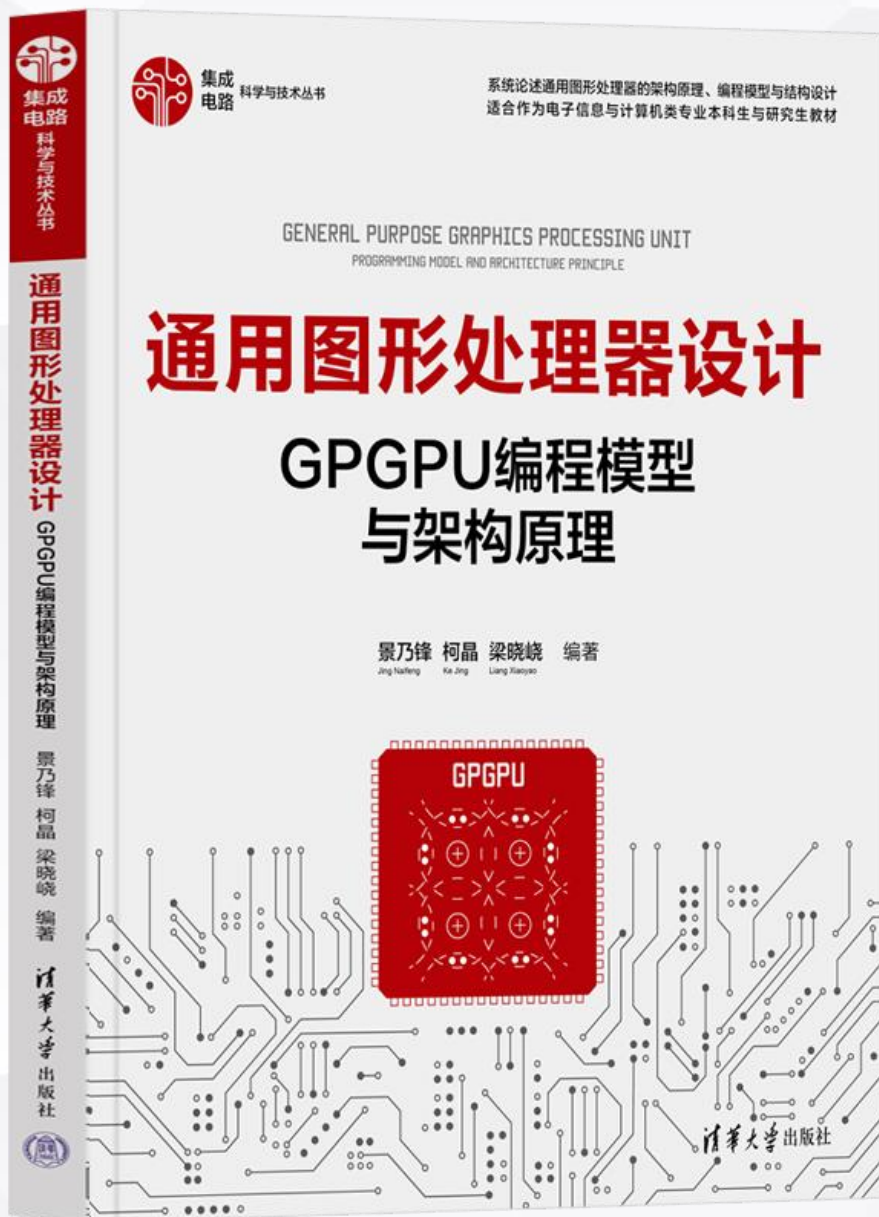
通用图形处理器设计

—GPGPU编程模型与架构原理

景乃锋

2026年春

饮水思源 · 爱国荣校



景乃锋、柯晶、梁晓峤 编著

清华大学出版社
2022.5月第一版

本书从GPGPU编程模型出发，系统介绍了异构计算的特点，GPGPU体系结构的工作原理、设计方法、关键技术和核心结构，让读者可以从系统和架构的视角，分析和理解GPGPU体系结构的编程模型和设计原理，进而去思考计算的方法和本质，把握未来高性能通用计算架构的发展方向。本书试图通过对国际前沿研究成果的介绍，解释GPGPU设计面临的挑战，促使读者进行更深入的思考，培养其创新思维能力。

目录



01

架构概述

02

编程模型

03

控制核心架构

04

存储架构

05

运算单元架构

06

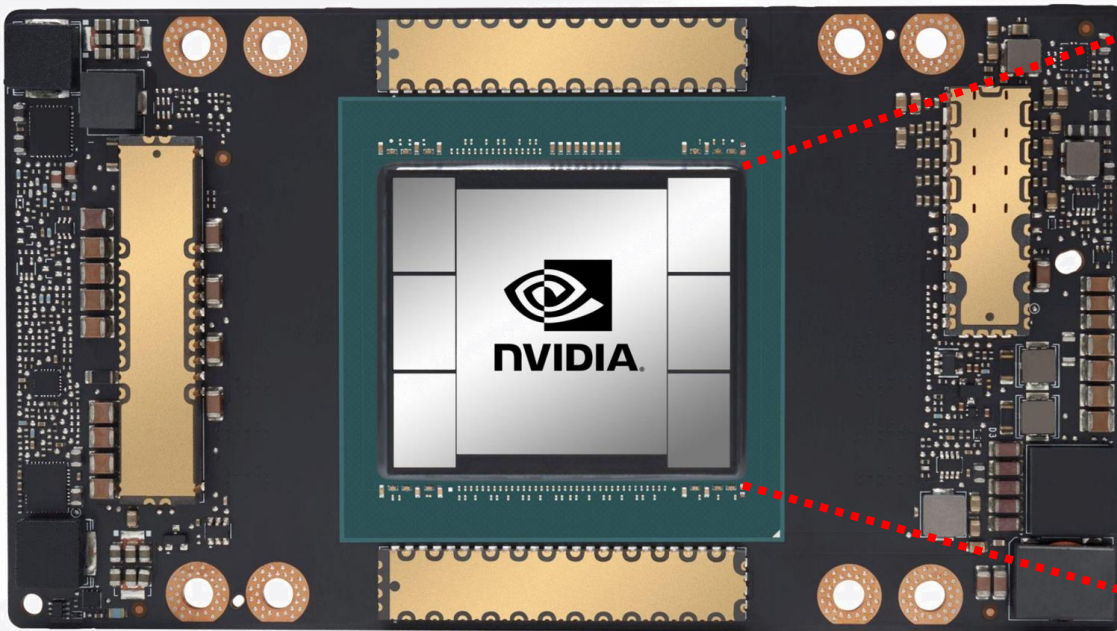
张量核心架构



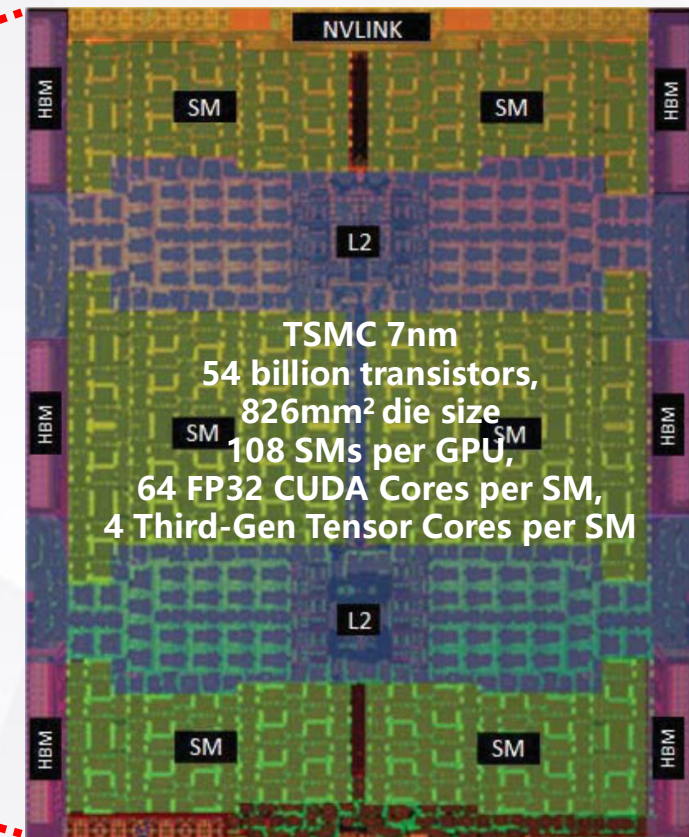
- GPU: Graphics Processing Unit, 图形处理器
- GPGPU: General-Purpose GPU, 通用图形处理器
- GPGPU originates from GPU, which is born for parallel processing
- Flynn terminology
 - SISD, SIMD, MISD, MIMD
- Parallelism
 - ILP, DLP, TLP
- Parallel architecture
 - Multi-core, cluster, WSE

Now, it becomes ... (NVIDIA, A100, 2020)

Whole A100 GPU Card with PCB, this is what you buy



A100 die photo under microscopy





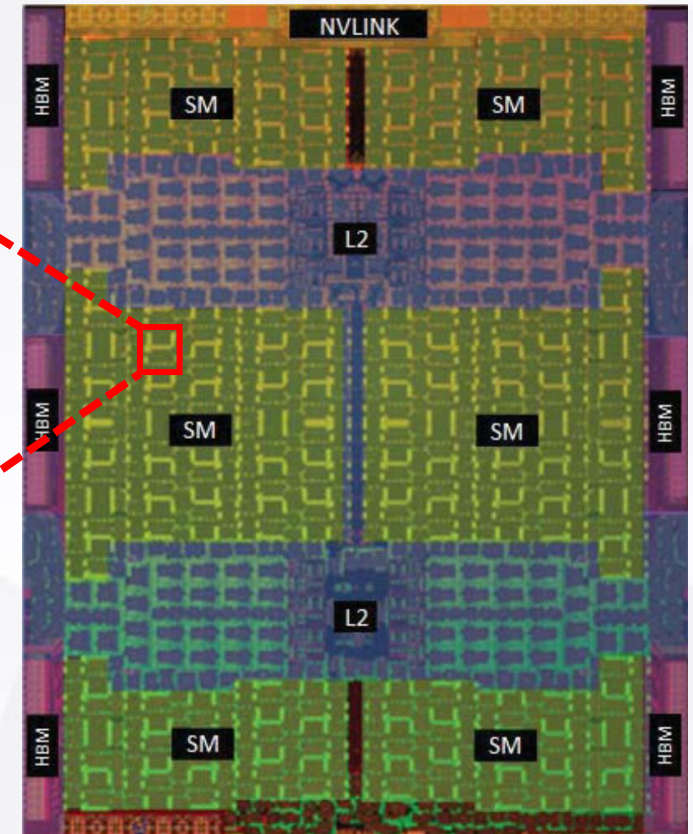
GPU Overview



Look inside



Streaming Multiprocessor (SM)
= Four sub-partition

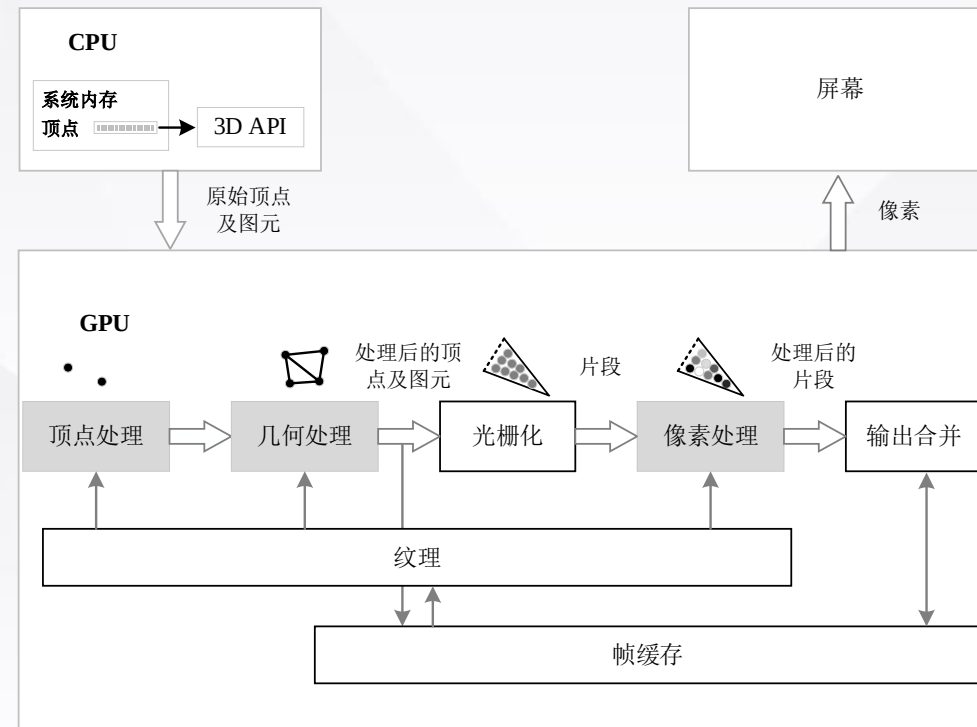




GPU history

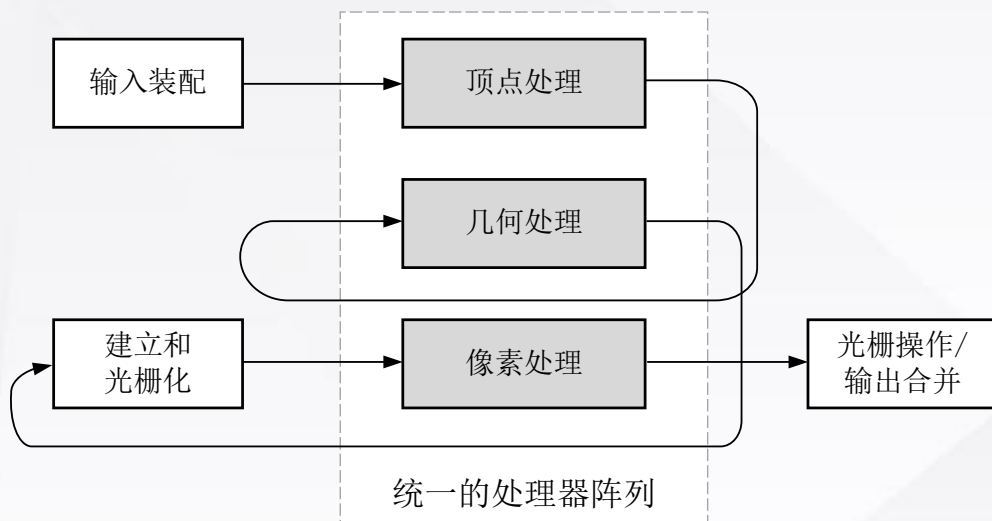
时间	类型	相关标准	代表产品	基本特征	意义
80年代	图形显示	CGA,VGA	IBM 5150	光栅生成器	最早图形显示控制器
80年代末	2D加速	GDI,DirectFB	S3 86C911	2D图元加速	开启2D图形硬件加速时代
90年代初	部分3D加速	OpenGL (1.1 ~ 4.1), DirectX (6.0~ 11)	3DLabs Glint300SX	硬件T&L	第一颗用于PC的3D图形加速芯片
90年代后期	固定管线		NVIDIA GeForce256	shader功能固定	首次提出GPU概念
2004~2010	统一渲染		NVIDIA G80	多功能shader	CUDA与G80一同发布
2011~至今	通用计算	CUDA,OpenCL1.2~ 2.0	NVIDIA TESLA	完成与图形处理无关的科学计算	NVIDIA正式将用于计算的GPU产品线独立出来

GPU逻辑图形流水线

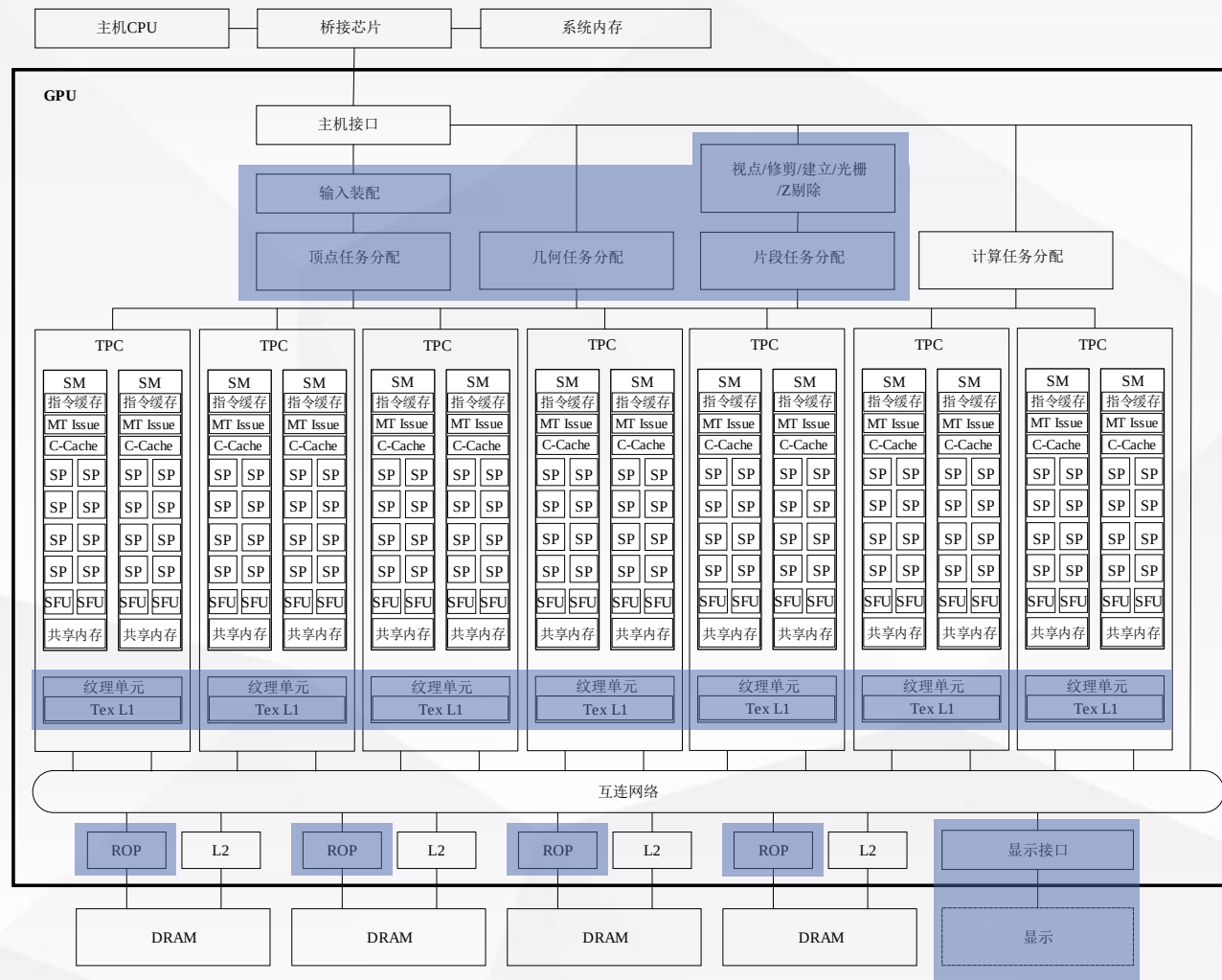




GPU vs. GPGPU



逻辑图形流水线向统一的处理器阵列结构映射



统一了图形和计算处理单元的GPU架构



Modern GPGPUs



Desktop

- NVIDIA
- AMD
- Intel



Embedded

- Imagination PowerVR
(once used by apple)
- Qualcomm Adreno
(once used by Samsung, XiaoMi)
- ARM Mali

NVIDIA Product	Tesla M2090	Tesla K40	Tesla M40	Tesla P100	Tesla V100	Tesla T4	Tesla A100
GPU	GF110	GK100	GM200	GP100	GV100	TU104	GA100
Architecture	Fermi	Kepler	Maxwell	Pascal	Volta	Turing	Ampere
SMs	16	15	24	56	80	40	108
CUDA Cores	512	2880	3072	3584	5120	2560	6912
Tensor Cores	NA	NA	NA	NA	640	320	432
GPU Boost Clock (MHz)	NA	810/875	1114	1480	1530	1590	1410
Peak FP32 (GFLOPS)	1332	5046	6844	10609	15670	8141	19490
Peak FP64 (GFLOPS)	666.1	1682	213.9	5304	7834	254.4	9746
Peak Tensor (TFLOPS, FP16)	NA	NA	NA	NA	125	65	312
Memory Interface	384-bit GDDR5	384-bit GDDR5	384-bit GDDR5	4096-bit HBM2	4096-bit HBM2	256-bit GDDR6	5120-bit HBM2e
Memory Size	6GB	Up to 12GB	Up to 24GB	16GB	16GB	16GB	40GB
TDP/Watts	250	235	250	300	300	70	250
Transistors (Billion)	3.0	7.1	8.0	15.3	21.1	13.6	54.2
Die Size (mm ²)	520	551	601	610	815	545	826
Manufacturing Process	40nm	28nm	28nm	16nm FinFET+	12nm FFN	12nm	7nm

目录



01

架构概述

02

编程模型

03

控制核心架构

04

存储架构

05

运算单元架构

06

张量核心架构

- 2001/2002, researchers see GPU as data parallel coprocessor
 - The GPGPU field is born
- 2007, NVIDIA releases CUDA 统一计算设备架构
 - CUDA – Compute Uniform Device Architecture
 - GPU shifts to GPGPU for computing
Graphic 编程 → Genral-Purpose编程
- 2008, Khronos releases OpenCL specification



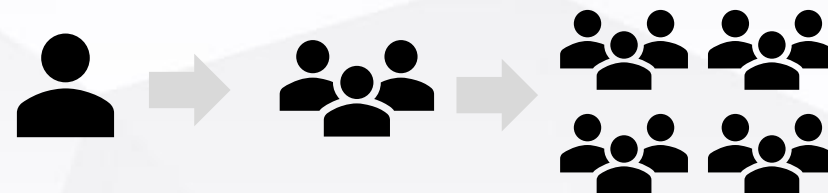
OpenCL

What a Programming Model should contain

- How to compute the wanted function 如何计算所需函数
- How to organize the memory to serve the computation 如何组织 memory 为计算服务
- How to map the function to the real hardware 如何将功能映射到实际硬件

Specifically, in a parallel architecture

- How to divide the workload 如何划分工作量
- How to communicate between the divided work 如何在分工之间进行沟通
- How to synchronize the divided work 如何同步已划分的工作

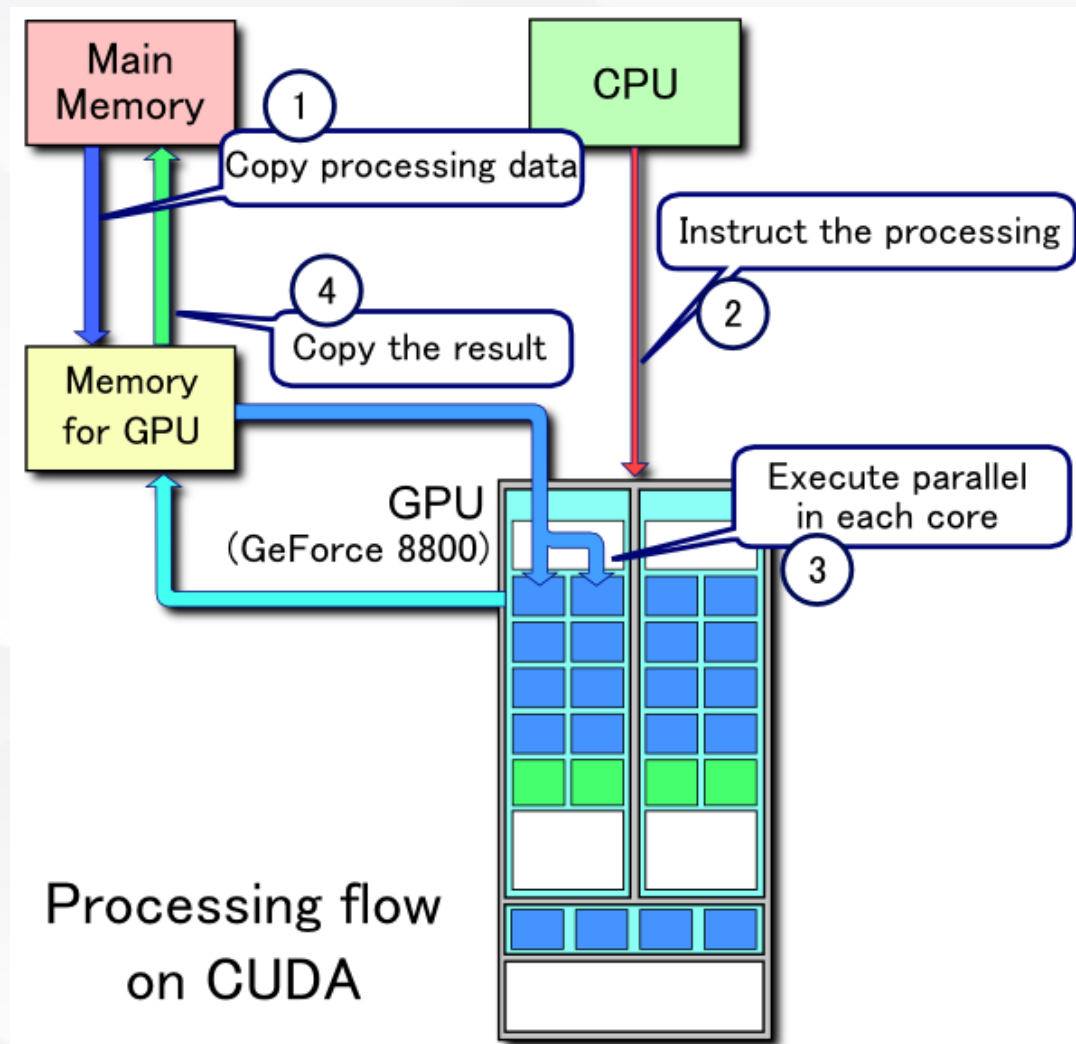
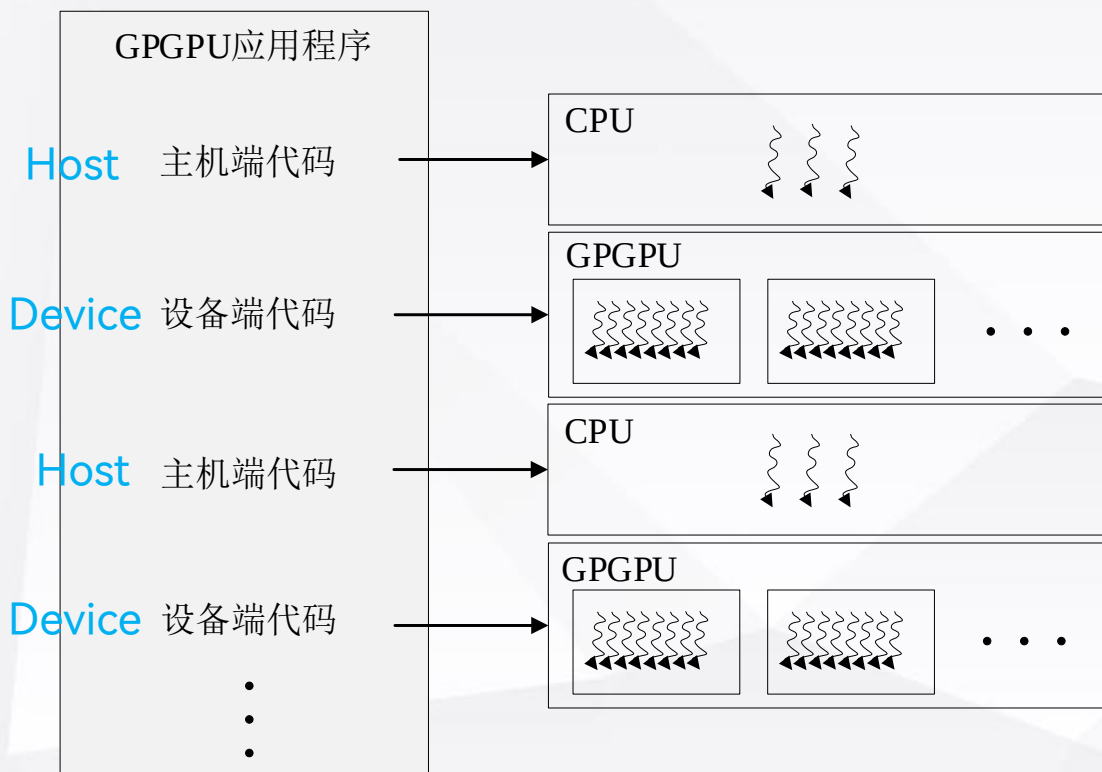


specify what each CUDA Core of each SM does at which moment

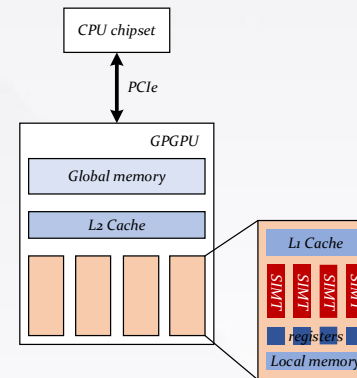
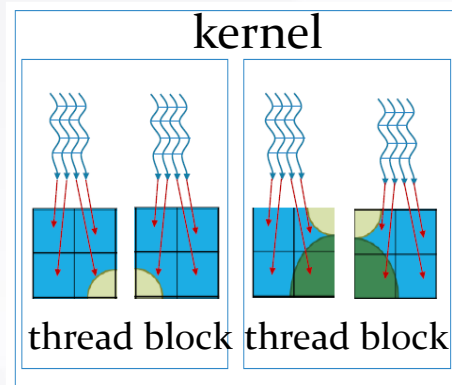
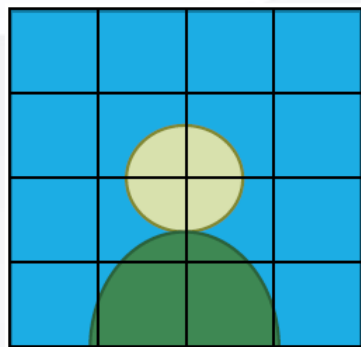
异构计算



Heterogeneous computing (programmer's view)



How to divide the workload



kernel $\rightarrow$ thread block $\rightarrow$ thread
 丢给 GPU $\rightarrow$ sm $\rightarrow$ SP

线程配置

CPU: __host__ $\rightarrow$ 指定在CPU上做

CPU invoke GPU by CPU通过内核（网格）、线程块、线程调用GPU

Name<<<dimGrid, dimBlock>>>(... parameters ...)

kernel (grid), thread block, thread

thread grid

CUDA: __device__ / __global__

$\rightarrow$ 指定在GPU上做

Grid里有多少个th bl th bl里分配多少个th
 $A = B * C \quad (8192 = 16 * 512)$

vect_mult<<< 16, 512>>>(n, a, b, c);
 or 8 1024



Thread Model

Grid, thread block, thread

Thread hierarchy

- `gridDim(x,y,z)`

- `blockDim(x,y,z)`

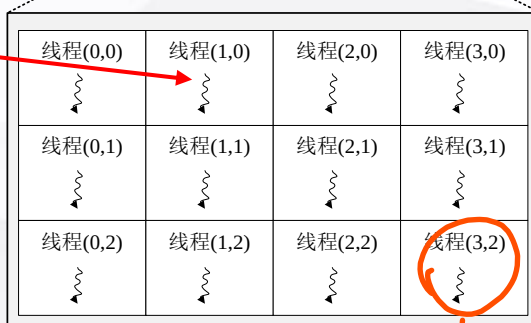
Data indexing

- `blockIdx(x,y,z)`

- `threadIdx(x,y,z)`

一个grid有几个block

一个block有几个thread



6x

3×2

$x=3$

$y=2$

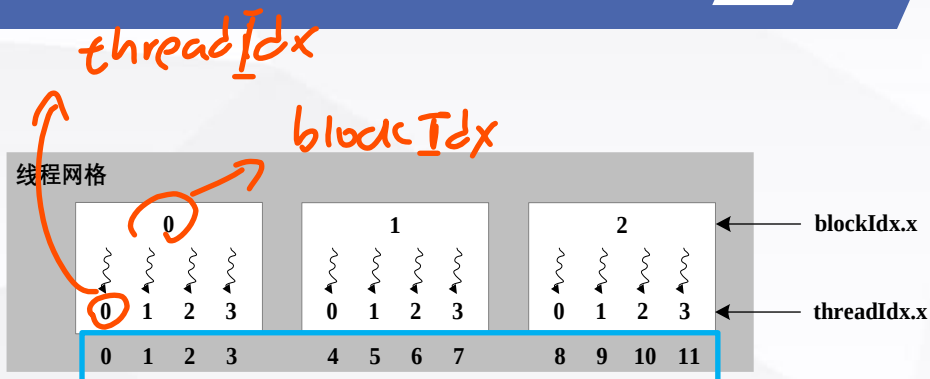
$z=1$

`gridDim(3,2,1)`

4×3

thread

$x=3$
 $y=2$



```
1 __global__ void kernel1 (int* A){
2   int index = threadIdx.x + blockIdx.x * blockDim.x;
3   A[index] = index;
4 }
```

kernel1结果: 0 1 2 3 4 5 6 7 8 9 10 11

Handwritten notes: 4, 第几个block*block大小+本地第几个, $11 = 3 + 2 \times 4$

```
5 __global__ void kernel2 (int* A){
6   int index = threadIdx.x + blockIdx.x * blockDim.x;
7   A[index] = blockIdx.x;
8 }
```

kernel2结果: 0 0 0 0 1 1 1 1 2 2 2 2

```
9 __global__ void kernel3 (int* A){
10  int index = threadIdx.x + blockIdx.x * blockDim.x;
11  A[index] = threadIdx.x;
12 }
```

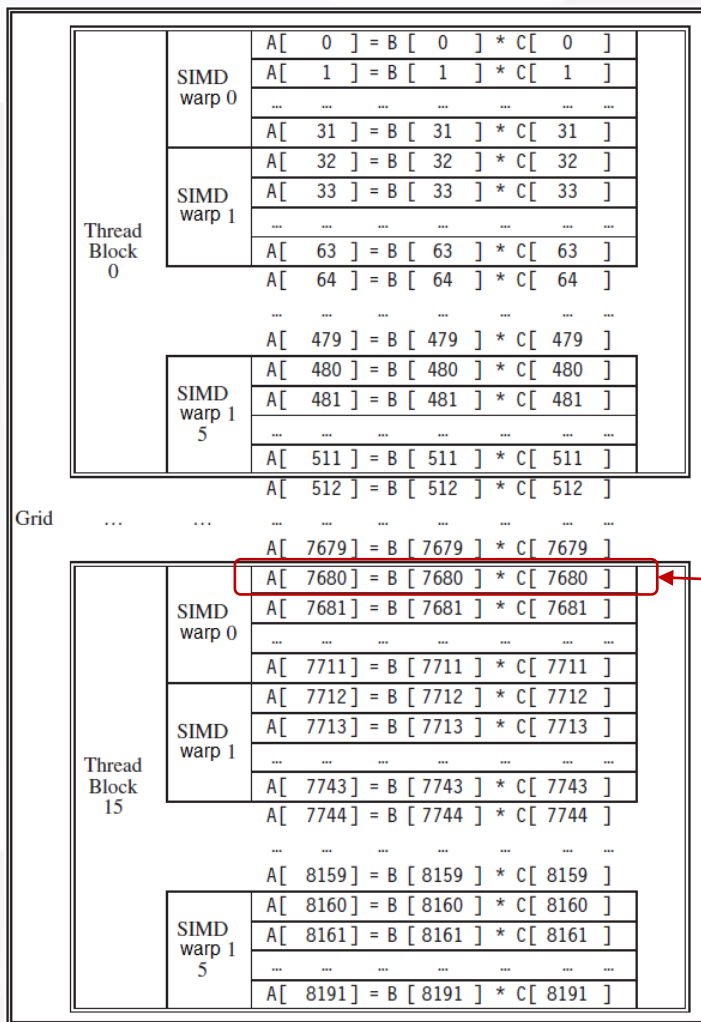
kernel3结果: 0 1 2 3 0 1 2 3 0 1 2 3



Thread Model



Mapping data to threads



// **host code** CPU端代码

// Name<<<dimGrid, dimBlock>>>(... parameter list ...)

$A = B * C$ (8192 = 16 * 512)

vect_mult<<< 16, 512>>>(n, a, b, c);

// **device code** GPU端代码

```
__device__ void vect_mult(int n, double *a, double *b, double *c)
{
```

```
    int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    if (i < n)
```

```
        a[i] = b[i] * c[i];
```

```
}
```

例如如果有8191个元素，则
最后一个线程可以根据这一句防
止越界

Grid: 8192 elements $\rightarrow$ 8192 threads

16 thread blocks with $8192/16=512$ threads per thread block

512 threads with $512/32=16$ **线程束** **warps** per thread block

如果没有512个独立的sp，只有32个独立的sp，就需要时分复用。每32个线程构成一个线程束(warp)，总共需要16个线程束

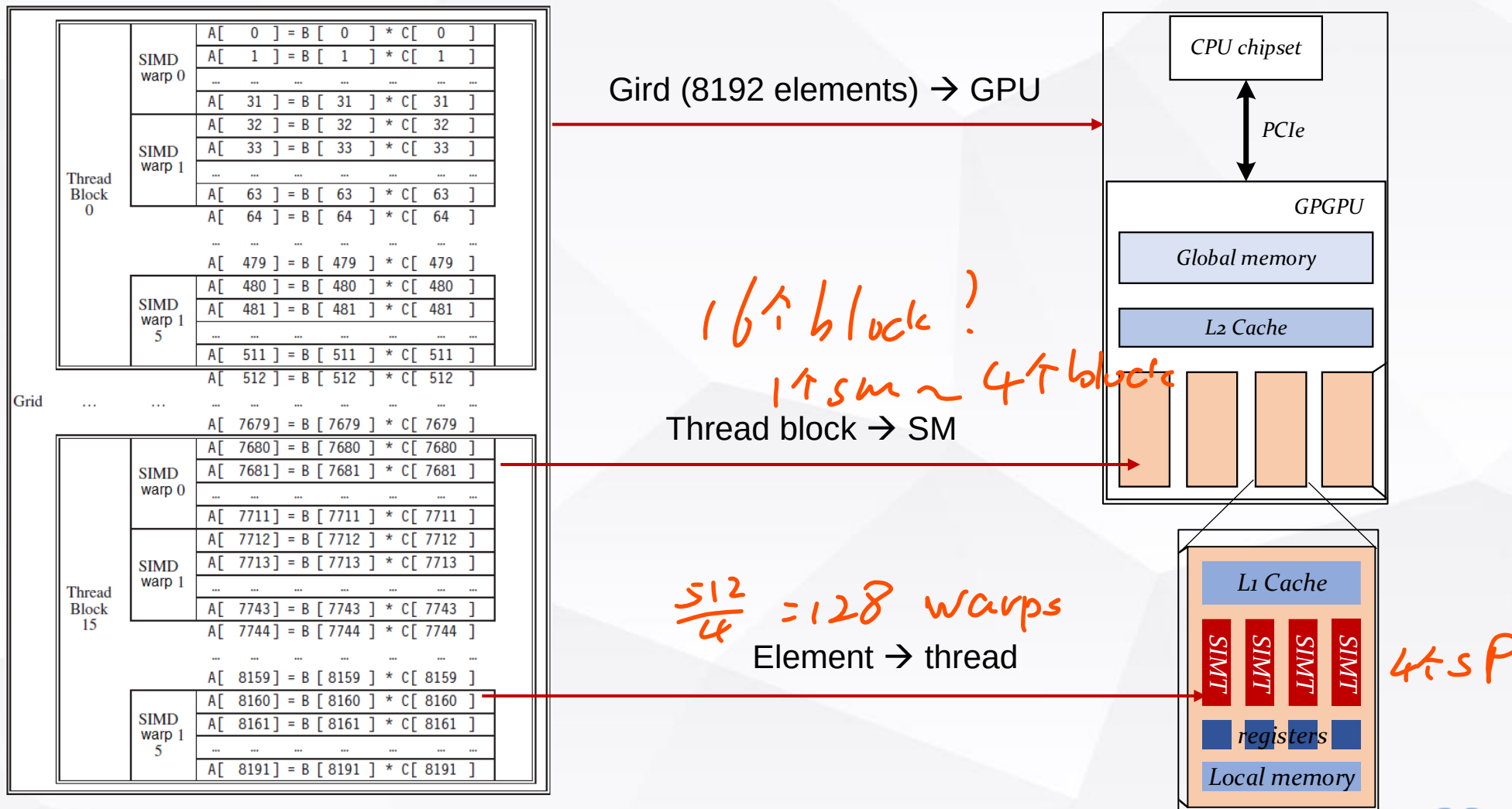




Thread Model



Thread mapping to hardware





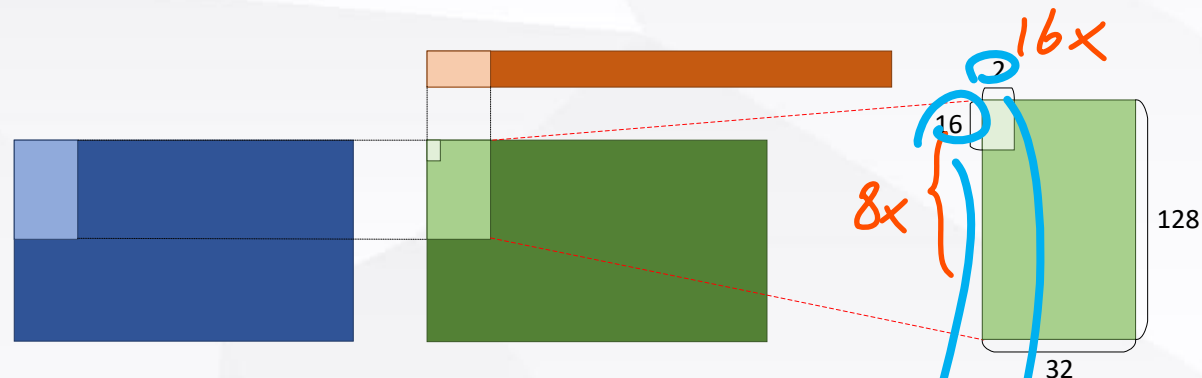
Another CUDA Example



Matrix-Vector Addition

- $C[i,j] = A[i,j] + B[i]$

矩阵 + 向量广播



How to map the task?

- Sliced into [32,128] (in size) tile per Thread Block (TB)
- (16,8)=128 Thread per TB
- Each Thread in TB computes [2,16] element

Device code

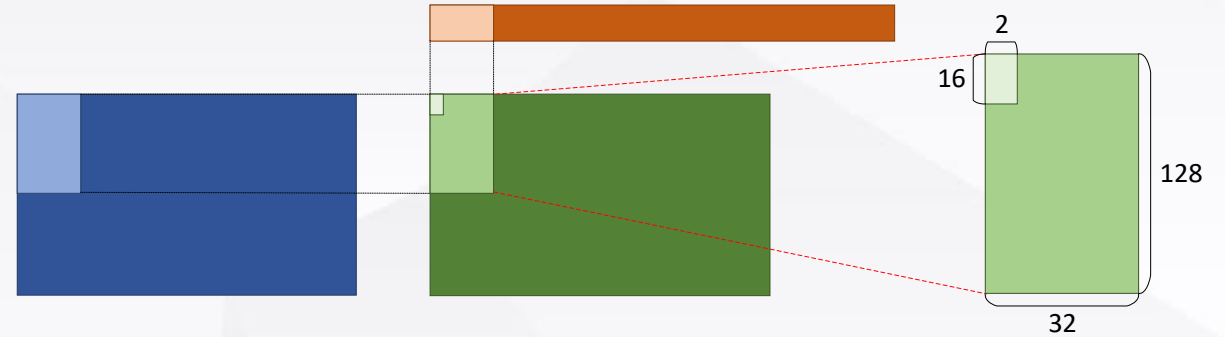
```
1  __global__ void matrixAddBiasLargeTile(float *matrix, const float *bias,
2      int numRows, int numCols)
3  {
4      int row_start = blockIdx.y * blockDim.y * 16 + threadIdx.y * 16;
5      int col_start = blockIdx.x * blockDim.x * 2 + threadIdx.x * 2;
6      for (int i = 0; i < 16; ++i) { // 16 rows
7          int row = row_start + i;
8          if (row < numRows) {
9              for (int j = 0; j < 2; ++j) { // 2 columns
10                 int col = col_start + j;
11                 if (col < numCols) {
12                     matrix[row * numCols + col] += bias[col];
13                 }
14             }
15         }
16     }
17 }
```





Matrix-Vector Addition

- $C[i,j] = A[i,j] + B[i]$



Host code

```
1  int numRows = 1024; // Number of rows in the matrix
2  int numCols = 1024; // Number of columns in the matrix
3  size_t matrixSize = numRows * numCols * sizeof(float);
4  size_t biasSize = numCols * sizeof(float); // Define the dimensions of the grid and blocks
5
6  .....
7
8  dim3 threadsPerBlock(16, 8); // Adjusted for fewer threads per block
9  dim3 blocksPerGrid((numCols + 31) / 32, // Each thread does 2 cols, so double the number of cols covered by one block
10                      (numRows + 127) / 128); // Each thread does 16 rows
11
12 // Launch the kernel
13 matrixAddBiasLargeTile<<<blocksPerGrid, threadsPerBlock>>>(d_matrix, d_bias, numRows, numCols);
```

$63 / 32 = 1 \times$ $(63 + 31) / 32 = 2$





Basic idea

- Match data requirement, reduce accesses to global

匹配数据需求，减少对全局的访问



Current types

- Register file
- Local memory
- Shared memory 程序员显示控制
- L1 data cache 之前学过的Cache，由硬件管理
- Global memory
- Constant memory
- Texture memory
- ...

将Cache动态转换为Shared Memory，如果程序员愿意主动优化

全部位于GDDR

GPI性能调优的重要资源

同一SM内部单独的存储空间，并非全局存储器

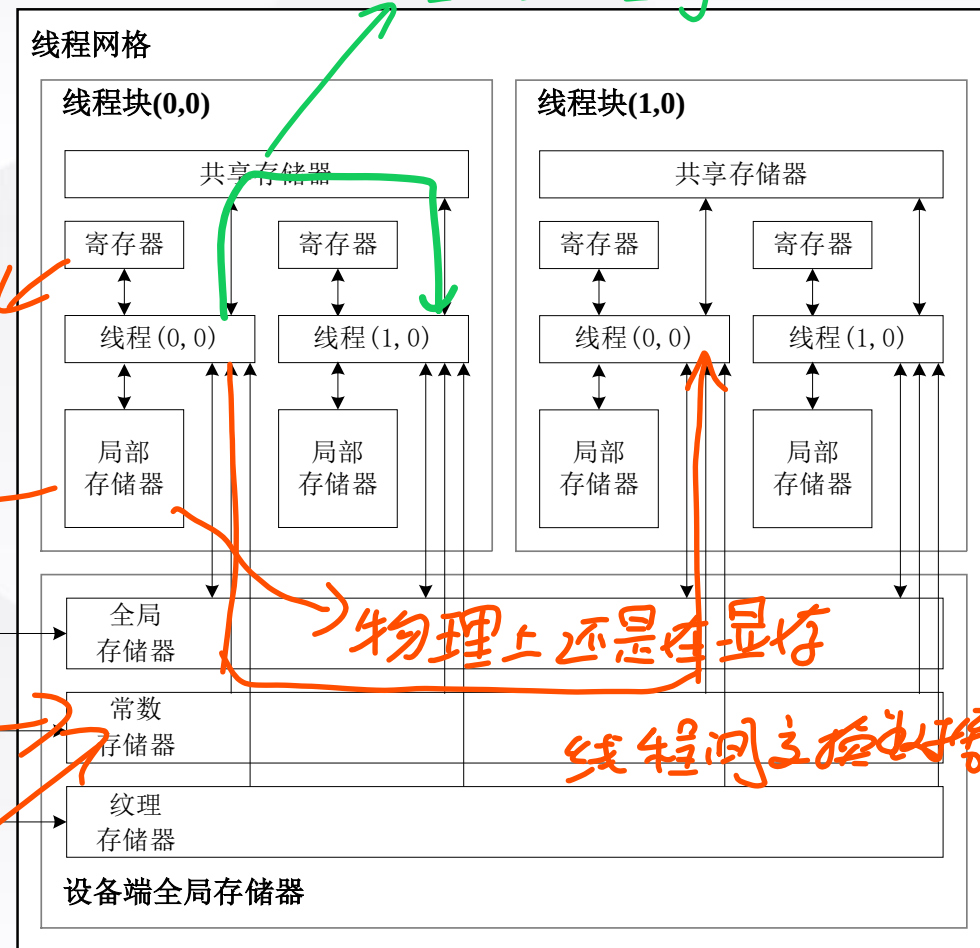
相邻线程间

thread级类似RISC R-type

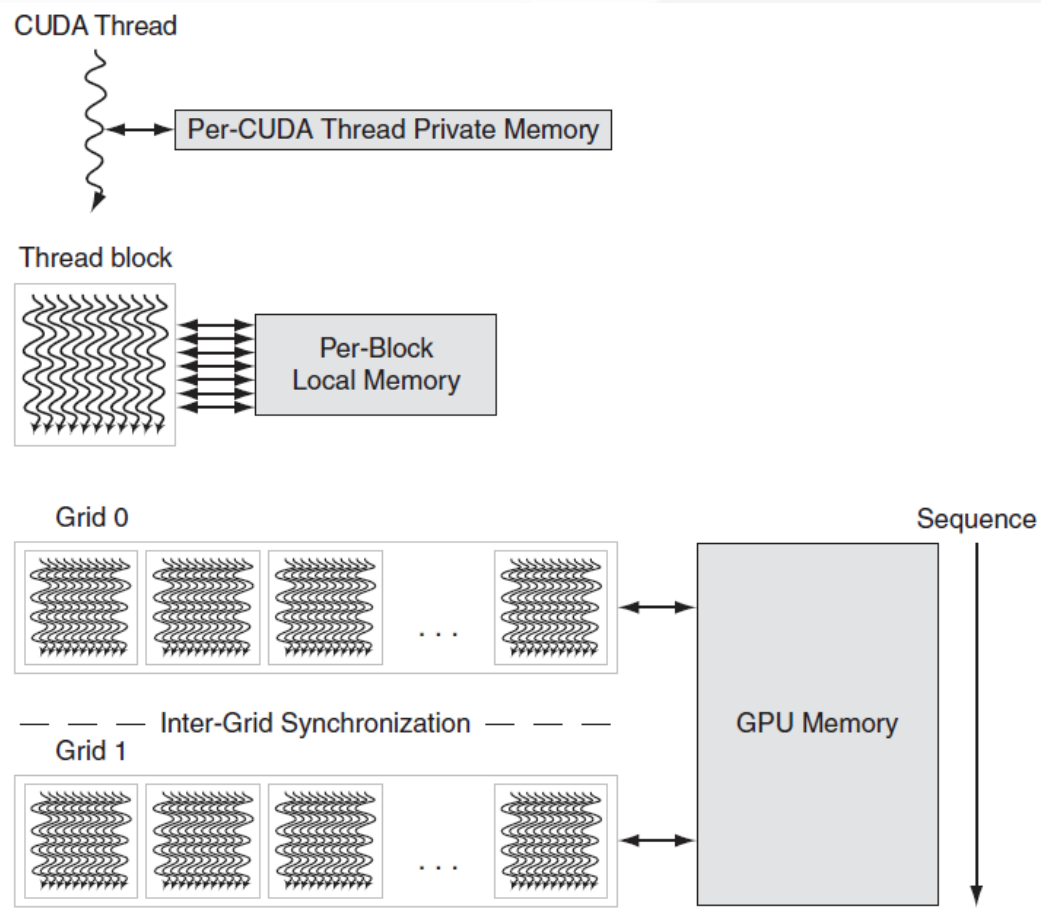
堆栈 thread级 main memory

物理上还是在显存

线程间交换数据



Related to thread model



描述	CUDA名称	OpenCL名称
所有线程（或work-items）均可访问	global memory	global memory
只读存储器	constant memory	constant memory
线程块（或work-group）内部线程访问	shared memory	local memory
单个线程（或work-item）可以访问	local memory	private memory



TLP is decided by

- # threads
- # thread blocks
- Size of register file
- Size of shared memory

Example:

GPU	Volta GV100
Compute Capability	7.0
Threads / Warp	32
Max Warps / SM	64
Max Threads / SM	2048
Max Thread Blocks / SM	32
Max 32-bit Registers / SM	65536
Max Registers / Block	65536
Max Registers / Thread	255 ¹
Max Thread Block Size	1024
FP32 Cores / SM	64
Ratio of SM Registers to FP32 Cores	1024
Shared Memory Size / SM	Configurable up to 96 KB

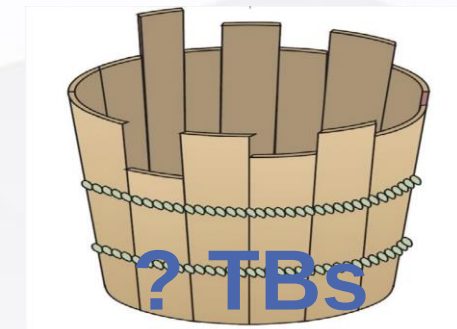
V100, 256KB RF, 96KB shared memory, 2048 threads per SM

$256\text{KB RF} / 2048 \text{ threads} = 32 \text{ regs per thread}$
More regs needed, then less threads

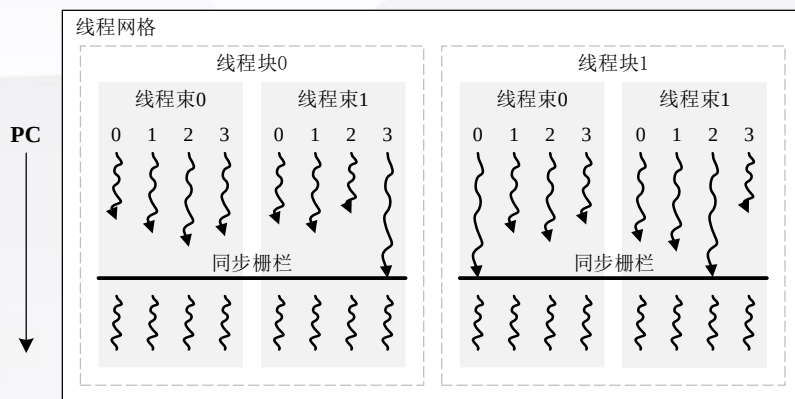
$96\text{KB shm} / 32 \text{ blocks} = 3\text{KB per block}$
More shm needed, then less blocks

One kernel with 64 regs/thread and 8KB shm, 128 threads/TB

$256\text{KB RF} / 64 \text{ reg/thread} = 1024 \text{ threads, i.e. 8 TBs}$
 $96\text{KB shm} / 8\text{KB shm} = 12 \text{ TBs}$



Sync between threads in a block scope



__syncthreads()及其对应的bar指令原理

Other scopes

- cudaDeviceSynchronize()
- cudaStreamSynchronize()

```

1  __global__ void block_mul(float* d_A, float* d_B, float* d_C)
2  {
3      __shared__ float Mds[BLOCK_SIZE][BLOCK_SIZE];
4      __shared__ float Nds[BLOCK_SIZE][BLOCK_SIZE];
5      int row = threadIdx.x + blockIdx.x * blockDim.x;
6      int col = threadIdx.y + blockIdx.y * blockDim.y;
7      int tx = threadIdx.x; int ty = threadIdx.y;
8      float P = 0;
9      for(int i = 0; i < N/BLOCK_SIZE; i++)
10     {
11         Mds[tx][ty] = d_A[row * N + ty + i * BLOCK_SIZE];
12         Nds[tx][ty] = d_B[col + (tx + i * BLOCK_SIZE) * K];
13         __syncthreads();
14         for(int j = 0; j < BLOCK_SIZE; j++)
15         {
16             P += Mds[tx][j] * Nds[j][ty];
17             __syncthreads();
18         }
19     }
20     d_C[row * K + col] = P;
21 }
    
```



SIMD vs. SIMT

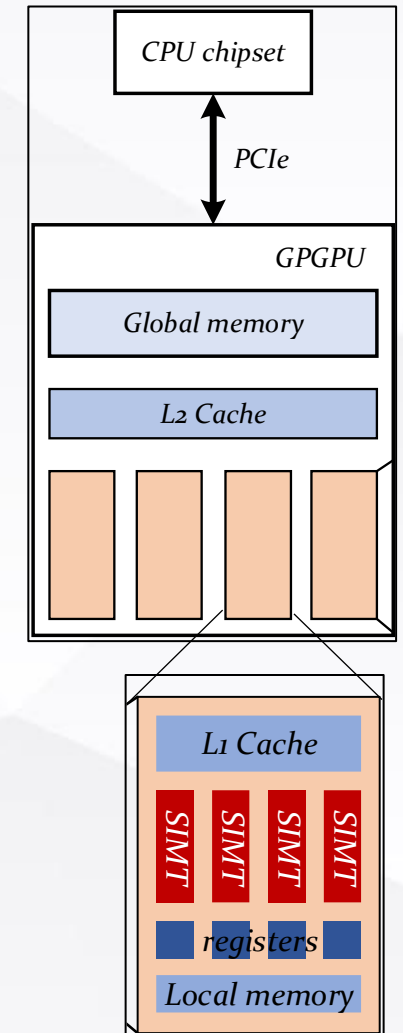
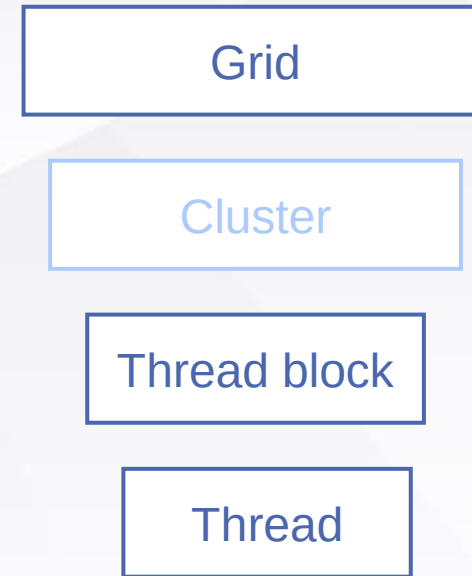
- SIMD: one thread + parallel OPs
- SIMT: many threads + scalar OPs
- Address scattering
- Thread divergence
- Number of instructions in ISA

C code	<pre>void vect_mult(int n, double *a, double *b, double *c) { for (int i = 0; i < n; ++i) a[i] = b[i] * c[i]; }</pre>
ARM (NEON)	<pre>void vect_mult(int n, uint32_t *a, uint32_t *b, uint32_t *c) { for(i = 0; i < n; i += 4) { uint32x4_t a4 = vld1q_u32(a+i); uint32x4_t b4 = vld1q_u32(b+i); uint32x4_t c4 = vmulq_u32(a4, b4); vst1q_u32(c+i, c4); } }</pre>
CUDA	<pre><u>__host__</u> int nblocks = (n+511)/512 // 512 threads per Thread Block vect_mult<<<nblocks, 512>>>(n, a, b, c); <u>__device__</u> void vect_mult(int n, double *a, double *b, double *c) { int i = blockIdx.x*blockDim.x + threadIdx.x; if (i < n) a[i] = b[i] * c[i]; }</pre>

Hierarchical thread model, **thread block**

- Cooperation between threads, e.g. reduction mapped on large-scaled hardware, reduce the communication cost
- Flexible number of threads for higher TLP, $\# \text{ thread blocks} > \# \text{ SMs}$
- Flexible scheduling on blocks
less aware of number of SMs in different GPUs

 When hardware continue to grow in scale, **thread block cluster** introduced in Hopper



**Programmability
hardware complexity
scalability**

Summary of Thread Model

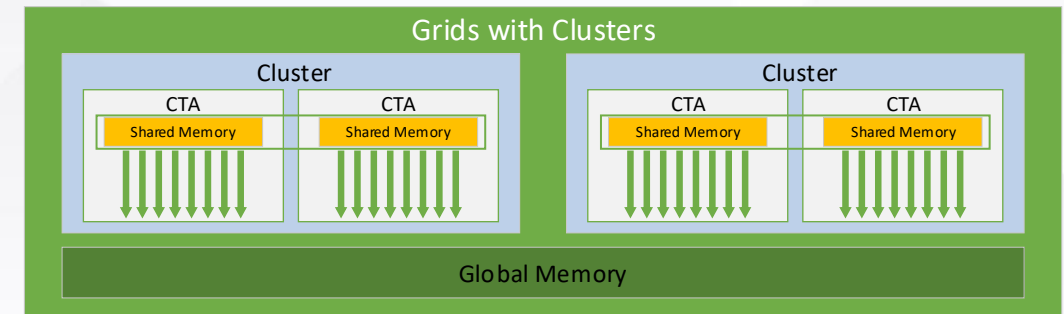
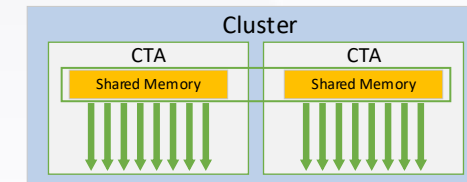
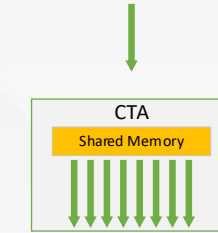
Heterogeneous Compute with GPU Thread Model

Operation	CUDA Cores / SFU / Tensor Cores
Thread	Streaming Processor (SP)
Thread block	Streaming Multiprocessor (SM)

Kernel (grid) — Whole GPU

Application — Host-device heterogeneous computing

$+/-/≤/≥/...$



CPU/memory

OpenMP



CUDA



- Designed for parallel computing with high performance
- Provide high level abstraction aligned with hardware

- Target at CPU
- Handle 10~100 threads
- No hierarchical programming model, easy to manage, but not well applicable to thousands of threads
- Global sync

- Target at GPU
- Handle 1000~10000 threads
- Hierarchical programming model like: Grid, Cluster, Thread Block, Thread
- Provide sync operations at different levels with different granularity